

Python Fundamentals

Introduction to Python

Theory:

- **Introduction to Python and its Features (simple, high-level, interpreted language).**

Python is a simple, high-level, interpreted programming language known for its easy syntax and readability. It supports multiple programming paradigms such as object-oriented, procedural, and functional programming. Python comes with a large standard library, automatic memory management, and strong community support, making it suitable for beginners and professionals.

- **History and evolution of Python.**

Python was created by Guido van Rossum and first released in 1991. It was designed to be simple and easy to read. Key milestones:

- **Python 2 (2000): Wider adoption, but later discontinued.**
- **Python 3 (2008): Major improvements in performance, security, and consistency.**

Today, Python is one of the most widely used languages in AI, data science, and web development.

- **Advantages of using Python over other programming languages.**

- ☐ **Easy to learn and write due to clean syntax.**
- ☐ **Cross-platform and works on Windows, macOS, and Linux.**
- ☐ **Extensive libraries (NumPy, Pandas, TensorFlow, Django).**
- ☐ **Strong community support and continuous updates.**
- ☐ **Ideal for AI, machine learning, data science, automation, and web development.**
- ☐ **Faster development time compared to languages like Java or C++**

- **Installing Python and setting up the development environment (Anaconda, PyCharm, or VS Code).**

You can use any of these:

- **Anaconda: Best for data science; includes Python + useful libraries + Jupyter Notebook.**
- **PyCharm: Professional IDE for Python projects.**
- **VS Code: Lightweight, fast editor with Python extensions. Steps:**

1. **Download Python or Anaconda from the official website.**
2. **Install and add Python to PATH.**
3. **Install a code editor (VS Code or PyCharm).**
4. **Verify installation using `python --version`.**

- **Writing and executing your first Python program.**

☐ **Open your editor or**

IDE. ☐ **Create a new**

file: `hello.py` ☐ **Write**

the code: `print("Hello,`

`Python!")`

☐ **Run the program:**

- **In terminal: `python hello.py` ☐ In IDE: click Run. [..\arbaz\pythone practical\first program.py](#)**

2. Programming Style

Theory:

- **Understanding Python's PEP 8 guidelines.**

PEP 8 is the official style guide for Python code. It helps developers write code that is clean, consistent, and readable.

Key points include proper indentation, naming styles, line length (max 79 characters), spacing, and organizing imports. Following PEP 8 makes code easier to understand and maintain across teams.

- **Indentation, comments, and naming conventions in Python.**

☐ **Indentation: Python uses 4 spaces per indentation level. Indentation is mandatory because it defines code blocks (loops, functions, conditions).**

☐ **Comments:**

- **Single-line: # This is a comment** ☐ **Multi-line (docstrings):**
- **"""This explains a function or module"""**

Comments should explain *why* code is written, not obvious details.

☐ **Naming Conventions:**

- **Variables & functions → snake_case**
- **Classes → PascalCase**
- **Constants → UPPER_CASE**
- **Avoid short, unclear names.**

- **Writing readable and maintainable code.**

Readable code is easy to understand; maintainable code is easy to update. Tips:

- **Use meaningful names for variables and functions.**
- **Keep functions short and focused on one task.**
- **Follow PEP 8 formatting for consistency.**
- **Add proper comments and docstrings.**
- **Avoid duplicate code; use functions or loops instead.**

- **Organize code into modules and follow logical structure.**

..\pythone practical\pep 8 copliant.py

3. Core Python Concepts

Theory:

- **Understanding data types: integers, floats, strings, lists, tuples, dictionaries, sets.**
 - ☐ **Integers: Whole numbers (e.g., 10, -5).**
 - ☐ **Floats: Decimal numbers (e.g., 3.14, 0.5).**
 - ☐ **Strings: Text enclosed in quotes (e.g., "Hello").**
 - ☐ **Lists: Ordered, changeable collections (e.g., [1, 2, 3]).**
 - ☐ **Tuples: Ordered but immutable collections (e.g., (1, 2, 3)).**
 - ☐ **Dictionaries: Key-value pairs (e.g., {"name": "John", "age": 20}).**
 - ☐ **Sets: Unordered collections of unique elements (e.g., {1, 2, 3}).**
- **Python variables and memory allocation.**

A variable in Python is created when you assign a

value to a name: `x = 10`

Python uses dynamic typing, so the variable type changes based on the value assigned. Memory is managed automatically through Python's memory manager and garbage collector, which handle storage and release of objects without manual effort. ..\pythone practical\input.py

- **Python operators: arithmetic, comparison, logical, bitwise.**

Python provides different types of operators for performing operations:

- **Arithmetic Operators:** +, -, *, /, %, //, ** **Used for mathematical operations.**
- **Comparison Operators:** ==, !=, >, <, >=, <= **Used to compare two values and return True/False.**
- **Logical Operators:** and, or, not **Used for combining conditional statements.**
- **Bitwise Operators:** &, |, ^, ~, <<, >> **Used to perform operations on binary (bit-level) data.**

[..\pythone practical\variable,datatype.py](#)

4. Conditional Statements

Theory:

- **Introduction to conditional statements:** if, else, elif.
- **Nested if-else conditions.**
- **if:** Executes a block of code when a condition is True.

```
if age >= 18:  
    print("Adult")
```

- **elif (else if):** Used to check multiple conditions when the previous if condition is False.

```
if marks >= 90:  
    print("A grade")  
elif marks >= 75:  
    print("B grade")
```

- **else:** Executes when none of the previous conditions are True.

```
if num > 0:  
    print("Positive")  
else:  
    print("Non-positive")
```

[..\pythone practical\statment.py](#)

[..\pythone practical\nestedif.py](#)

These statements help control program flow based on logical decisions.

- **Nested if-else Conditions**

A nested if-else means placing one if-else statement inside another.

It is used when decisions require multiple levels of checking.

Example:

```
if score >=
50:   if score
    >= 90:

    print("Excellent")
else:
    print("Good") else:
    print("Fail")
```

Nested conditions are useful for handling complex decision-making, but should be used carefully to keep code readable.

[..\pythone practical\statment2.py](#)

[..\pythone practical\grandcalculatestatment.py](#)

5 Looping (For, While)

Theory:

- **Introduction to for and while loops.**

Loops in Python are used to repeat a block of code multiple times.

for loop: Used when you want to iterate over a sequence (like a list, string, or range).

while loop: Used when you want to repeat code as long as a condition remains True.

- **How loops work in Python.**

for loop

The for loop goes through each item in a sequence automatically:

```
for i in range(5):  
    print(i)
```

Here, the loop runs 5 times (0 to 4).

while loop

The while loop continues until the condition becomes False:

```
count = 1  
while count <= 5:  
    print(count)  
    count += 1
```

This loop runs until count becomes greater than 5.

[..\pythone practical\table.py](#)

□ Using loops with collections (lists, tuples, etc.).

Loops are very useful for working with collections like lists, tuples, strings, sets, and dictionaries.

[..\pythone practical\for loop.py](#)

[..\pythone practical\patterusnignested.py](#)

6. Generators and

Iterators Theory:

- **Understanding how generators work in Python.**

Generators are special functions that generate values one at a time instead of storing all values in memory.

They use the yield keyword to produce a sequence lazily (on demand).

Generators are memory-efficient, fast, and ideal for working with large datasets or infinite sequences.

Difference Between `yield` and `return`

Feature	<code>yield</code>	<code>return</code>
Purpose	Produces a value and pauses the function	Ends the function and returns a value
Execution	Function state is saved	Function terminates completely
Output	Generator object (sequence of values)	Single value
Usage	Used in generators	Used in normal functions

- **Understanding iterators and creating custom iterators.**

An iterator is an object that can be looped over using `next()`. It must implement two methods:

- `__iter__()` → **returns the iterator object**
- `__next__()` → **returns the next value**

Theory:

- **Defining and calling functions in Python.**
- **Function arguments (positional, keyword, default).**
- **Scope of variables in Python.**
- **Built-in methods for strings, lists, etc.**

[..\pythone practical\itratore.py](#)

7. Functions and Methods

Theory:

- **Defining and calling functions in Python.**

Functions are reusable blocks of code defined using the def keyword. A function is executed only when called.

- **Function arguments (positional, keyword, default).**

- ☐ **Positional Arguments: Passed in the correct order.**

```
def add(a, b): print(a + b)
add(3, 5)
```

- ☐ **Keyword Arguments: Passed using the parameter name.**

```
add(b=5, a=3)
```

- ☐ **Default Arguments: Have a predefined value if not provided.**

```
def greet(name="User"):
    print("Hello", name)
```

[..\pythone practical\string.py](#)

- **Scope of variables in Python.**

Scope decides where a variable can be accessed.

- **Local Scope: Variables created inside a function; accessible only within that function.**
- **Global Scope: Variables declared outside functions; accessible throughout the program.**

Python follows the LEGB Rule (Local → Enclosing → Global → Built-in) for variable lookup.

[..\pythone practical\stringusingloop.py](#)

- **Built-in methods for strings, lists, etc.**

String Methods

- **upper() – convert to uppercase**
- **lower() – convert to lowercase**
- **split() – split into list**
- **replace() – replace substring**

List Methods

- **append() – add element**
- **sort() – sort list**
- **remove() – remove element**
- **insert() – insert at position**

Dictionary Methods

- **keys() – get all keys**
- **values() – get all values**
- **items() – key-value pairs**

Set Methods

- **add()** – add element
- **clear()** – remove all elements

[..\pythone practical\string methond.py](#)

8. Control Statements (Break, Continue, Pass)

Theory:

- Understanding the role of break, continue, and pass in Python loops.

1. break

- **Used to stop a loop immediately.**
- **When break is executed, the loop ends even if the condition is still true.**

[..\pythone practical\break.py](#)

2. continue

- **Skips the current iteration and moves to the next iteration of the loop.**
- **The loop does not stop; only that iteration is skipped.**

[..\pythone practical\continuestatment.py](#)

3. pass

- **A null statement; it does nothing.**
- **Used as a placeholder when a statement is required but you don't want any action.**

9. String Manipulation

Theory:

- **Understanding how to access and manipulate strings.**

Strings in Python are sequences of characters and can be accessed using indexing.

- **Positive indexing starts from 0.**
- **Negative indexing starts from -1 (from the end).**
- **Basic operations: concatenation, repetition, string methods (upper(), lower(), etc.).**

Joining two or more strings using +:

`"Hello " + "World"`

Repetition

Repeating a string using *:

`"Hi" * 3 # "HiHiHi"`

[..\pythone practical\lenth.py](#)

Common String Methods

- **upper() – converts to uppercase**
- **lower() – converts to lowercase**
- **strip() – removes spaces**
- **replace(old, new) – replaces text**
- **split() – breaks string into list**
- **find() – returns index of substring**

[..\pythone practical\string methond.py](#)

String Slicing

Slicing allows extracting parts of a string using [start:end:step].

Examples:

```
s = "Python"
s[0:3] # 'Pyt'
s[2:]  # 'thon'
s[:4]  # 'Pyth'
s[::-1] # reverse
str
```

[..\pythone practical\slicing.py](#)

10. Advanced Python (map(), reduce(), filter(), Closures and Decorators)

Theory:

- How functional programming works in Python.
 - Functions can be passed as arguments, returned from other functions, and used to process data efficiently.
 - Functional programming helps write clean, concise, and readable code.
-

- Using map(), reduce(), and filter() functions for processing data.

Using map(), filter(), and reduce()

1. map()

□ Applies a function to each item of a sequence and returns a map object.

```
nums = [1, 2, 3]
squared = list(map(lambda x: x**2, nums)) # [1, 4, 9]
```

[..\pythone practical\mapf.py](#)

2. filter()

□ Filters items from a sequence based on a condition.

```
nums = [1, 2, 3, 4]
even = list(filter(lambda x: x % 2 == 0, nums)) # [2, 4]
```

[..\pythone practical\filter.py](#)

3. reduce()

- **Applies a function cumulatively to items of a sequence, reducing it to a single value.**
- **Available in functools module.**

```
from functools import
reduce
nums = [1, 2, 3, 4]
sum_all = reduce(lambda x, y: x + y, nums) #
```

[..\pythone practical\redeuse.py](#)

- **Introduction to closures and decorators.**

Closures

- **A closure is a function enclosed within another function that remembers the outer function's variables even after the outer function has finished execution.**

Example:

```
def outer(x):
    def inner(y):
        return x + y
    return inner
```

```
add5 = outer(5)
add5(3) # 8
```

Decorators

- **Decorators are functions that modify the behavior of another function without changing its code.**
- **Used for logging, authentication, or measuring execution time.**

Example:

```
def decorator(func):  
def wrapper():  
    print("Before function")  
    func()  
        print("After function")  
    return wrapper
```

```
@decorator  
def greet():  
    print("Hello")
```

```
greet()
```